



**"RANKED 45
IN INDIA**

#University Category



**WORLD
UNIVERSITY
RANKINGS**

**NO.1 PVT. UNIVERSITY IN
ACADEMIC REPUTATION IN INDIA**



**ACCREDITED GRADE 'A'
BY NAAC**



**PERFECT SCORE OF 150/150 AS A TESTAMENT
TO EXCEPTIONAL E-LEARNING METHODS**

1



Applied Machine Learning

Unit 07 – Lecture 23: Partitioning and Density-Based Clustering

Tofik Ali

School of Computer Science, UPES Dehradun

Autumn 2026

Choose k representatives. Assign. Repeat.

Topic focus: **what a “cluster centre” is**

Repository: <https://github.com/tali7c/Applied-Machine-Learning>

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 10.; James et al., *An Introduction to Statistical Learning with Python*, Ch. 12.; Müller and Guido, *Introduction to Machine Learning with Python*, §3.5.; Ester, Kriegel, Sander and Xu, “A density-based algorithm for discovering clusters (DBSCAN),” KDD 1996.

Quick Links

Partitioning

k-means by hand

Convergence

Choosing *k*

Assumptions

k-medoids / PAM

CLARA

DBSCAN

One global eps

Summary

Agenda

Partitioning: the problem and the objective

A complete *k*-means run, by hand

Why it converges, and to what

Choosing *k*

What *k*-means assumes, and where it breaks

k-medoids and PAM

CLARA: Clustering Large Applications

Density-based clustering: DBSCAN

DBSCAN's weakness: one global ϵ

Summary

How this lecture works

Every idea appears twice

1. **By hand** — seven points and a pen for *k*-means, eleven points and a pen for DBSCAN. Small enough that every number can be checked.
2. **In code** — the same numbers out of `sklearn.cluster`, agreeing exactly, so you can see the library is doing precisely this and nothing magic.

Commit before you look

Seven slides ask you to predict something before it is revealed. Write your guess down. Being wrong on paper is how the idea sticks.

Two calculations carry the lecture: **a complete *k*-means run on seven points, iteration by iteration, with the objective written down after every half step, and eleven points labelled core, border or noise arithmetically.** Those are the examinable ones.

Three kinds of seed — this lecture uses all three

One arbitrary seed

0. Any KMeans start that is not an experiment, every CLARA run, every contamination draw.

Dataset seeds

`random_state=7` (six blobs), 11 (four clean), 3 (unequal sizes). These fix the *shape* — and the shape is the argument.

Sweep seeds

for `s` in `range(200)`. The disagreement between runs *is* the result. Collapse it and you delete the lecture.

Pin `n_init` as well as `random_state`

`random_state` fixes *where* the starts come from; `n_init` fixes *how many*. Its default is "auto" — 1 for `k-means++`, 10 for `init="random"` — and it changed in scikit-learn 1.4. Write `KMeans(n_clusters=6)` alone and you will not reproduce these figures.

Where we are

The last lecture

Hierarchical clustering: you supply nothing, you get a tree, you choose *k* afterwards. It is deterministic and it never undoes a merge — and it needs $O(n^2)$ memory, so it stops at a few tens of thousands of rows.

Today: the other bargain

Partitioning. You supply *k*. You get one partition, fast, on data a hierarchy could not touch — and you get a *different* answer every run.

And then: refuse the bargain

Density-based. Do not supply *k*, do not assume round clusters, and let the method say “this point belongs to nothing”.

The partitioning problem, stated precisely

Given

n objects and an integer *k*. **Find** a partition into exactly *k* non-empty groups that optimises some criterion.

The criterion for *k*-means: within-cluster sum of squares

$$\text{WCSS} = \sum_{j=1}^k \sum_{x \in C_j} \|x - c_j\|_2^2, \quad c_j = \frac{1}{|C_j|} \sum_{x \in C_j} x$$

Three names for the same number

WCSS, *inertia* (scikit-learn's `inertia_`), and the *distortion*. All the same sum.

Note the square

k-means minimises *squared* Euclidean distance. That is not a detail — it is exactly why the optimal representative of a cluster is its **mean**, and why one far-away point costs so much.

Why not just try all the partitions?

n	k	number of partitions $S(n,k)$
7	2	63
10	3	9330
25	3	1.41198e+11
50	5	7.40096e+32
100	3	8.58963e+46

Stirling numbers of the second kind

$$S(n, k) = \frac{1}{k!} \sum_{i=0}^k (-1)^i \binom{k}{i} (k-i)^n.$$

With $k = 2$ it is just $2^{n-1} - 1$. On seven points that is 63 and we will check every one of them.

k-means is NP-hard

Even for $k = 2$ in general dimension, and even for $d = 2$ with general k . So we do not solve it. We run a heuristic, and we are honest that it is one.

But the number *is* finite — hold that thought, it is half the convergence proof.

Lloyd's algorithm, in eight lines

Input : *n* points, an integer *k*, initial centroids *c*₁ ... *c*_{*k*}

Output: an assignment of every point to one of *k* clusters

repeat

 # ASSIGNMENT step

 for each point *x*: assign *x* to the cluster *j* minimising $||x - c_j||$

 # UPDATE step

 for each cluster *j*: *c*_{*j*} = mean of the points now assigned to *j*

until no point changes cluster

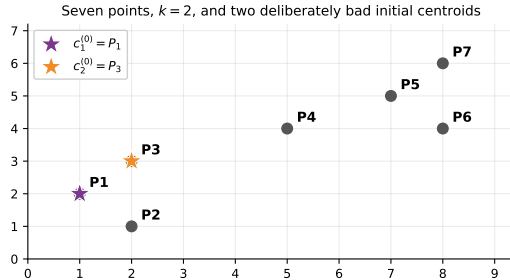
Two steps, alternating

Fix the centres, optimise the assignment. Fix the assignment, optimise the centres. Neither step ever needs to look at the other's problem.

What is *not* in the loop

No *k*-selection, no convergence tolerance, no learning rate. The only free choice is the **initial centroids** — and that choice decides the answer.

Seven points, $k = 2$, and a deliberately bad start



$P_1(1, 2)$ $P_2(2, 1)$ $P_3(2, 3)$ $P_4(5, 4)$ $P_5(7, 5)$ $P_6(8, 4)$ $P_7(8, 6)$

Initial centroids taken from the data: $c_1^{(0)} = P_1 = (1, 2)$ and $c_2^{(0)} = P_3 = (2, 3)$ — both on the left, on purpose.

Predict first #1 — where will this end up?

Commit to an answer

Both starting centroids are in the left-hand clump. The four points on the right are nowhere near either of them.

Does *k*-means recover $\{P_1, P_2, P_3\}$ and $\{P_4, P_5, P_6, P_7\}$ from that start?
How many iterations?

Predict first #1 — where will this end up?

Commit to an answer

Both starting centroids are in the left-hand clump. The four points on the right are nowhere near either of them.

Does *k*-means recover $\{P_1, P_2, P_3\}$ and $\{P_4, P_5, P_6, P_7\}$ from that start?
How many iterations?

Yes — and in **three** iterations, with exactly **one** point changing cluster on the way.

The interesting part is *which* point moves, and *when*. At iteration 1 the split is $\{P_1, P_2\}$ against the other five. Watch P_3 .

By hand, iteration 1 — the assignment step

$c_1 = (1, 2)$, $c_2 = (2, 3)$. One Euclidean distance per point per centroid; assign to the smaller.

point	coord	d(.,c1)	d(.,c2)	cluster
P1	(1.0, 2.0)	0.0000	1.4142	C1
P2	(2.0, 1.0)	1.4142	2.0000	C1
P3	(2.0, 3.0)	1.4142	0.0000	C2
P4	(5.0, 4.0)	4.4721	3.1623	C2
P5	(7.0, 5.0)	6.7082	5.3852	C2
P6	(8.0, 4.0)	7.2801	6.0828	C2
P7	(8.0, 6.0)	8.0623	6.7082	C2
clusters: C1 = {P1,P2} C2 = {P3,P4,P5,P6,P7}				

$d(P_2, c_1) = \sqrt{1^2 + 1^2} = \sqrt{2} = 1.4142$ against $d(P_2, c_2) = \sqrt{0^2 + 2^2} = 2$. Every entry is checkable in your head.

WCSS after this assignment = $2 + 121 = 123.0000$.

By hand, iteration 1 — the update step

New centroids are plain averages

$$c_1 = \frac{1}{2}((1+2), (2+1)) = (1.5, 1.5)$$

$$c_2 = \frac{1}{5}((2+5+7+8+8), (3+4+5+4+6)) \\ = \left(\frac{30}{5}, \frac{22}{5}\right) = (6.0, 4.4)$$

$$c_1 = \text{mean}\{P_1, P_2\} = (1.5, 1.5)$$

$$c_2 = \text{mean}\{P_3..P_7\} = (6.0, 4.4)$$

$$\text{WCSS after assignment} = 123.0000$$

$$\text{WCSS after update} = 32.2000$$

Check the drop by hand

C_1 contributes $0.5 + 0.5 = 1.0$. C_2 contributes $17.96 + 1.16 + 1.36 + 4.16 + 6.56 = 31.2$. Total **32.2**, down from 123.

Nothing was reassigned. The objective fell by 90.8 purely because the centres moved to the means. **That is what the update step is for.**

Predict first #2 — does anything move now?

Commit to an answer

The centroids are now $c_1 = (1.5, 1.5)$ and $c_2 = (6.0, 4.4)$. P_3 sits at $(2, 3)$ and is currently in C_2 .

Does P_3 stay in C_2 ? Compute the two distances before you look.

Predict first #2 — does anything move now?

Commit to an answer

The centroids are now $c_1 = (1.5, 1.5)$ and $c_2 = (6.0, 4.4)$. P_3 sits at $(2, 3)$ and is currently in C_2 .

Does P_3 stay in C_2 ? Compute the two distances before you look.

$$d(P_3, c_1) = \sqrt{0.25 + 2.25} = \mathbf{1.5811} \quad d(P_3, c_2) = \sqrt{16 + 1.96} = \mathbf{4.2379}$$

P_3 **moves to** C_1 , and no other point changes.

P_3 was only in C_2 because c_2 *started* on top of it. One update step later, c_2 has been dragged away to $(6, 4.4)$ by the four right-hand points, and P_3 is abandoned.

By hand, iterations 2 and 3

Iteration 2: P_3 crosses over

point	coord	d(.,c1)	d(.,c2)	c1
P1	(1.0, 2.0)	0.7071	5.5462	C1
P2	(2.0, 1.0)	0.7071	5.2498	C1
P3	(2.0, 3.0)	1.5811	4.2379	C1 <--
P4	(5.0, 4.0)	4.3012	1.0770	C2
P5	(7.0, 5.0)	6.5192	1.1662	C2
P6	(8.0, 4.0)	6.9642	2.0396	C2
P7	(8.0, 6.0)	7.9057	2.5612	C2

WCSS after assignment = 16.7400
 $c1 = \text{mean}\{P1, P2, P3\} = (1.6667, 2.0000)$
 $c2 = \text{mean}\{P4..P7\} = (7.0000, 4.7500)$
 WCSS after update = 11.4167

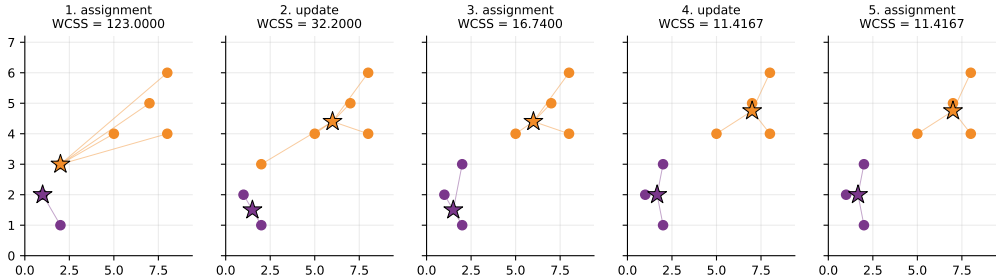
Iteration 3: nothing moves

P1	0.6667	6.6002	C1
P2	1.0541	6.2500	C1
P3	1.0541	5.2974	C1
P4	3.8873	2.1360	C2
P5	6.1192	0.2500	C2
P6	6.6416	1.2500	C2
P7	7.4907	1.6008	C2

no point changed cluster
 CONVERGED after 3 iterations

$$c_1 = \left(\frac{5}{3}, 2\right), c_2 = \left(7, \frac{19}{4}\right), \text{WCSS} \\ = \frac{8}{3} + \frac{35}{4} = \frac{137}{12} = 11.4167$$

Drawn: the whole run in five panels



Stars are centroids; the thin lines are the distances the objective adds up. Panels alternate assignment, update, assignment, update, assignment — and the last panel is identical to the one before it, which is what “converged” means.

In code: the library does exactly that

```
km = KMeans(n_clusters=2, init=np.array([P7[0], P7[2]]),  
            n_init=1, algorithm="lloyd").fit(P7)  
print(km.labels_, km.cluster_centers_, km.inertia_, km.n_iter_)
```

```
sklearn labels      [0 0 0 1 1 1 1]      hand labels  [0 0 0 1 1 1 1]  
sklearn centres     [[1.6667, 2.0], [7.0, 4.75]]  
hand centres        [[1.6667, 2.0], [7.0, 4.75]]  
sklearn inertia_    11.416667      hand WCSS 11.416667      match to 1e-12: True  
iterations n_iter_ = 3
```

And it is the global optimum here — checked exhaustively

All $2^6 - 1 = 63$ two-way partitions of seven points were scored. The best is WCSS = **11.4167** with $\{P_1, P_2, P_3\}$ against $\{P_4, P_5, P_6, P_7\}$ — the answer Lloyd found. **On seven points. Do not generalise from that.**

The convergence argument, in three lines

1. The assignment step cannot increase WCSS

Each point is moved to the centroid nearest it. Its own contribution $\min_j \|x - c_j\|^2$ can only fall or stay. Sum over points.

2. The update step cannot increase WCSS

For a fixed set S , the point minimising $\sum_{x \in S} \|x - z\|^2$ is $z = \bar{x}$, the mean. Setting c_j to the mean of C_j is therefore optimal for that cluster.

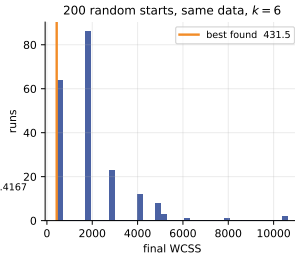
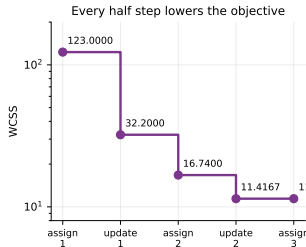
(Proof: $\sum_{x \in S} \|x - z\|^2 = \sum_{x \in S} \|x - \bar{x}\|^2 + |S| \|\bar{x} - z\|^2$, and the second term is minimised at $z = \bar{x}$. $\square$)

3. There are finitely many assignments

$S(n, k)$ of them. A sequence that never increases and never repeats a strictly better value cannot run for ever. **So the algorithm halts.**

Measured: the objective after every half step

```
step      WCSS
assign 1  123.0000
update 1  32.2000    <-- centres move to the means
assign 2  16.7400    <-- P3 crosses over
update 2  11.4167    <-- centres move again
assign 3  11.4167    <-- nothing changes: stop
strictly non-increasing: True
```



Predict first #3 — how bad can a bad start be?

Commit to a number

Six well-separated Gaussian blobs, 600 points, $k = 6$. Run k -means twenty times from **uniformly random** initial centroids, changing only the seed. The best run reaches WCSS 431.54 and recovers the blobs exactly (ARI 1.000).

What is the *worst* of the twenty, as a multiple of the best?

Predict first #3 — how bad can a bad start be?

Commit to a number

Six well-separated Gaussian blobs, 600 points, $k = 6$. Run *k*-means twenty times from **uniformly random** initial centroids, changing only the seed. The best run reaches WCSS 431.54 and recovers the blobs exactly (ARI 1.000).

What is the *worst* of the twenty, as a multiple of the best?

Worst = 5010.58, which is **11.6**× the best.

Twelve distinct final WCSS values out of twenty runs.

Every one of those twelve is a *local* optimum: no single point can be reassigned and no centre moved to improve it. The algorithm did nothing wrong. It converged. It just converged somewhere else.

The twenty runs, in full

seed	inertia	ARI	seed	inertia	ARI	seed	inertia	ARI
0	431.5370	1.0000	7	5010.5833	0.5825	14	1719.5194	0.7697
1	431.5370	1.0000	8	1884.4493	0.7673	15	2737.9794	0.7718
2	431.5370	1.0000	9	1885.0220	0.7677	16	431.5370	1.0000
3	2740.4734	0.7787	10	1720.3550	0.7697	17	1719.5313	0.7697
4	1720.5136	0.7693	11	431.5370	1.0000	18	431.5370	1.0000
5	431.5370	1.0000	12	1883.8297	0.7673	19	431.5370	1.0000
6	2737.2417	0.7713	13	431.5370	1.0000			

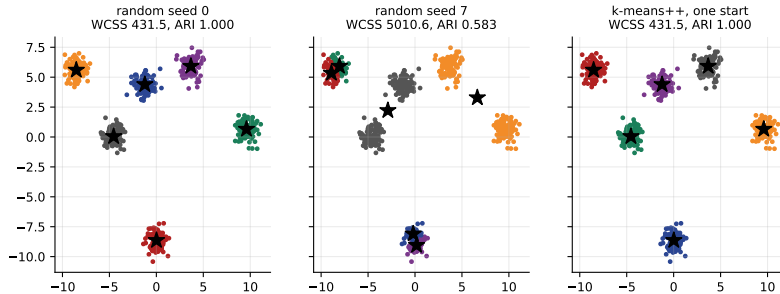
Read the ARI column

Nine runs got it perfectly right. Ten got roughly 0.77 — two true blobs merged and one split. One got 0.58.

The failure has a shape

It is always the same failure: two initial centres land in the same blob, so that blob is split and some other pair is merged. Nothing can repair it, because moving one centre across the gap makes WCSS worse first.

Drawn: three runs on the same data



Left: seed 0, the right answer. Middle: seed 7, two centres in one blob and three real blobs fused. Right: one *k*-means++ start, no restarts, straight to the best solution.

The fix: *k*-means++ and *n_init*

k-means++ seeding (Arthur and Vassilvitskii, 2007)

Pick c_1 uniformly at random. Then pick each further centre with probability proportional to $D(x)^2$, the squared distance to the nearest centre already chosen. Far-away points are far more likely to be picked, so the centres spread out. Guarantees $\mathbb{E}[\phi] \leq 8(\ln k + 2) \phi_{\text{OPT}}$.

init	mean WCSS	best	worst	% reaching best	(200 single starts)
random	1907.6245	431.5370	10621.5284	32.0	
<i>k</i> -means++	437.9858	431.5370	1721.3003	99.5	

<i>n_init</i> = 1 (random)	: mean	1836.85	worst	10621.53	% at best	35.0
<i>n_init</i> = 3 (random)	: mean	948.52	worst	2737.75	% at best	62.5
<i>n_init</i> = 10 (random)	: mean	463.74	worst	1719.53	% at best	97.5
<i>n_init</i> = 25 (random)	: mean	431.54	worst	431.54	% at best	100.0
<i>n_init</i> = 1 (<i>k</i> -means++)	: mean	431.54	worst	431.54	% at best	100.0

What that costs, and what to actually do

```
n_init = 1 0.0009 s   n_init = 10 0.0059 s   n_init = 50 0.0227 s
(exactly 1x, 10x and 50x the starts -- the seconds are machine-dependent)
```

The practical rule

`n_init` restarts cost `n_init` times as much and you keep the run with the lowest `inertia_`. It is the cheapest insurance in machine learning: 10× of a five-millisecond fit.

Read the release notes

scikit-learn's `n_init` default became "auto", which is **1** for `k-means++` and **10** for `init="random"`. If you set `init="random"` and also `n_init=1`, you have opted into the 32% row above.

Never report a *k*-means result from a single random start, and always record the seed.

Two heuristics, and neither is a criterion

The elbow

WCSS falls monotonically with *k* and reaches zero at *k* = *n*, so you cannot minimise it. Instead look for the *k* where the *rate* of improvement collapses.

Two standard automations: largest perpendicular distance to the chord joining the ends (“kneedle”), and largest collapse in the ratio of successive drops.

The silhouette

For point *i* with mean in-cluster distance *a*(*i*) and mean distance *b*(*i*) to the nearest *other* cluster,

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \in [-1, 1].$$

Average over all points; pick the *k* that maximises it.

The silhouette *can* be maximised, which makes it feel like a criterion. It is not. It rewards compact well-separated clusters — that is, it rewards exactly the shape *k*-means assumes.

Predict first #4 — how many clusters in iris?

Commit to an answer

Iris, 150 flowers, four measurements, standardised. The WCSS values for $k = 1 \dots 8$ are

600.0, 222.4, 139.8, 114.1, 90.8, 80.0, 71.6, 62.9.

What does the elbow say? What does the silhouette say? And iris has *three* species.

Predict first #4 — how many clusters in iris?

Commit to an answer

Iris, 150 flowers, four measurements, standardised. The WCSS values for $k = 1 \dots 8$ are

600.0, 222.4, 139.8, 114.1, 90.8, 80.0, 71.6, 62.9.

What does the elbow say? What does the silhouette say? And iris has *three* species.

Chord elbow: $k = 3$. Drop-ratio elbow: $k = 2$. Silhouette: $k = 2$ (0.5818).

Best ARI: $k = 3$ (0.6201).

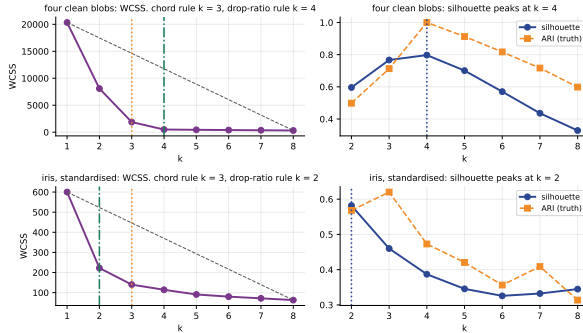
Two automations of the same heuristic disagree with each other, before you even bring in the silhouette.

The numbers behind that

iris, standardised			four clean blobs				
k	WCSS	silhouette	k	WCSS	drop	silhouette	
1	600.0000	--	1	20345.30	--	--	
2	222.3617	0.5818	2	8095.80	12249.50	0.5960	
3	139.8205	0.4599	3	1880.33	6215.47	0.7668	
4	114.0925	0.3869	4	493.25	1387.08	0.7973	
5	90.8076	0.3455	5	442.57	50.68	0.7008	
6	80.0225	0.3257	6	397.59	44.98	0.5702	
summary			chord	ratio	silhouette	ARI-best	truth
four clean blobs			3	4	4	4	4
iris			3	2	2	3	3

On the clean blobs the drop from 1387 to 50.7 is unmistakable and three of the four rules say 4. On iris nothing agrees with anything. **Two of the three iris species genuinely overlap** — the data really does look like two clusters.

Drawn: both criteria on both data sets



Left column: WCSS with the chord (dashed grey) and the two elbow verdicts. Right column: silhouette against the ARI with the true labels.

The honest version: uniform noise has an “elbow” too

500 points drawn **uniformly at random** in the unit square. There is no cluster structure whatsoever.

k	WCSS	silhouette
1	80.6344	--
2	49.7154	0.3592
3	31.2614	0.3839
4	20.3558	0.4060
5	16.6794	0.3853
6	13.7743	0.3917
7	11.3759	0.3895

<-- silhouette peaks here

The curve is smooth, and yet the silhouette still has a maximum, at $k = 4$. ***k*-means always returns *k* clusters**, whether or not there are any. Neither the elbow nor the silhouette will tell you the answer is “none”.

Three assumptions hidden in one formula

$$\text{WCSS} = \sum_j \sum_{x \in C_j} \|x - c_j\|^2$$

Spherical

A cluster is described by *one point*, so the boundary between two clusters is the perpendicular bisector: cells of a Voronoi diagram, always **convex**.

Similar size

The sum is over points, so a big cluster contributes more terms and pulls harder. *k*-means prefers to **split a big cluster** rather than leave a small one alone.

Similar spread

A wide cluster and a tight one at the same distance are treated identically. The criterion has no notion of local density.

These are not bugs. They are the **definition** of what *k*-means is looking for. If your clusters are not like that, it will still return *k* of them, and they will be wrong.

Measured: four data sets it cannot handle

data	<i>k</i>	<i>k</i> -means ARI	sizes found
two moons	2	0.274	195,205
concentric circles	2	-0.002	207,193
two long thin bands	2	-0.001	399,401
very unequal sizes	3	0.288	208,176,96 (truth 400,40,40)

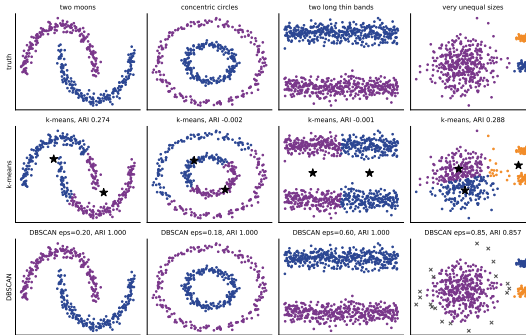
Look at the sizes

Every single answer is close to an equal split, because that is what minimising a sum of squares over a convex partition wants. On the last row the truth is 400/40/40 and *k*-means returns 208/176/96 — **it cut the big cluster into three.**

-0.002 and -0.001

An ARI at or below zero means the answer is no better than a random partition. On the two bands *k*-means returns a left half and a right half; the truth is a top band and a bottom band.

Drawn: the truth, k -means, and (spoiler) DBSCAN



Top: the truth. Middle: k -means with the correct k , centroids starred. Bottom: DBSCAN — ARI 1.000, 1.000, 1.000, 0.857 on the same four sets. We come back to that in the second half.

And the mean itself is not robust

The same seven points, plus **one** extra point at (12, 9).

k-means	without	the outlier:	centres	[[1.6667, 2.0], [7.0, 4.75]]	sizes	[3, 4]
k-means	with	the outlier:	centres	[[2.5, 2.5], [8.75, 6.0]]	sizes	[4, 4]
k-medoid	without	the outlier:	medoids	['P1', 'P5']	at	[[1, 2], [7, 5]]
k-medoid	with	the outlier:	medoids	['P1', 'P5']	at	[[1, 2], [7, 5]]
					E =	7.8929
					E =	14.2960

One point did two things

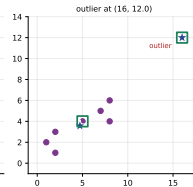
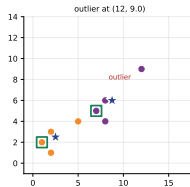
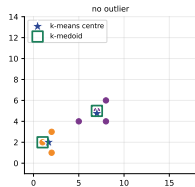
It moved the right-hand centre by **2.1506** — and it **changed the partition**: P_4 was dragged out of the right-hand cluster into the left one, giving 4/4 instead of 3/4.

The medoid did not move

A medoid is an *observation*. It cannot drift; it can only jump to another observation. P_1 and P_5 before, P_1 and P_5 after.

How far one point can move the mean

outlier at	k-means: right centre	shift	sizes	k-medoids: right medoid
none	(7.0000, 4.7500)	0.0000	3,4	P5 (7, 5)
(8, 6.0)	(7.2000, 5.0000)	0.3202	3,5	P5 (7, 5)
(10, 7.5)	(7.6000, 5.3000)	0.8139	3,5	P5 (7, 5)
(12, 9.0)	(8.7500, 6.0000)	2.1506	4,4	P5 (7, 5)
(16, 12.0)	(16.0000, 12.0000)	11.5569	1,7	OUT (16, 12)
(40, 30.0)	(40.0000, 30.0000)	41.5519	1,7	OUT (40, 30)



Beyond about (16,12) *both* methods give up a whole cluster to the single outlier — *k-medoids* is more robust, not immune.

Change one word in the objective

k-means

$$\sum_j \sum_{x \in C_j} \|x - c_j\|_2^2$$

c_j is the **mean** — a computed point, usually not in the data. Needs a vector space and squared Euclidean distance.

k-medoids

$$E = \sum_j \sum_{x \in C_j} d(x, o_j)$$

o_j is the **medoid** — an actual object of the data set. d is *any* dissimilarity. Not squared.

Three consequences, all of them the point of the method

1. The representative is a real object, so it is interpretable — “this customer is the archetype”.
2. Any dissimilarity works: edit distance, Jaccard, Gower, a precomputed matrix. There is no mean of two DNA sequences; there is a medoid.
3. An unsquared distance to a real point is far less sensitive to outliers than a squared distance to a movable mean.

PAM: Partitioning Around Medoids

BUILD : pick the object minimising total dissimilarity to everything;
then repeatedly add the object that reduces *E* the most, until *k*

SWAP : repeat

- for every medoid *o_i* and every non-medoid *o_h*:
 - compute the change in *E* if *o_i* were replaced by *o_h*
 - if the best change is negative: perform that swap
 - else: stop

The cost per SWAP iteration

$k(n - k)$ candidate swaps, each needing every object re-examined: $O(k(n - k)^2)$ per iteration.
That single expression is the reason CLARA exists.

And $O(n^2)$ memory

PAM works from the dissimilarity matrix, exactly like hierarchical clustering, so it inherits the same memory wall.

By hand: score all $\binom{7}{2} = 21$ medoid pairs

{P1,P2}	E =	26.5784	
{P1,P3}	E =	22.7526	
{P1,P4}	E =	11.6700	
{P1,P5}	E =	7.8929	<-- best
{P1,P6}	E =	9.2426	
{P1,P7}	E =	9.8482	
{P2,P3}	E =	22.7526	
{P2,P4}	E =	12.2558	
{P2,P5}	E =	8.4787	
{P2,P6}	E =	9.8284	
{P2,P7}	E =	10.4340	

{P3,P4}	E =	12.2558
{P3,P5}	E =	8.4787
{P3,P6}	E =	9.8284
{P3,P7}	E =	9.9907
{P4,P5}	E =	14.7055
{P4,P6}	E =	15.2913
{P4,P7}	E =	15.2913
{P5,P6}	E =	22.1468
{P5,P7}	E =	22.1468
{P6,P7}	E =	24.4853

$$E(\{P_1, P_5\}) = 0 + 1.4142 + 1.4142 + 2.2361 + 0 + 1.4142 + 1.4142 = \mathbf{7.8929}$$

Twenty-one numbers is small enough to brute-force. At $n = 100$ and $k = 5$ it is 75,287,520 — which is why PAM does a local search instead.

By hand: the SWAP step from a bad start

```
start with medoids {P1, P2}, E = 26.5784
step 1: best swap  P2 -> P5    dE = -18.6855    E = 7.8929
step 2: best swap  P1 -> P2    dE = +0.5858    E = 8.4787
           no swap improves E -- STOP
final medoids: {P1, P5}    E = 7.8929
```

Reading step 2

The *best* of the ten remaining candidate swaps still makes *E* worse, by 0.5858. That is the stopping condition: **no single swap improves the objective**, so we are at a local optimum of the swap neighbourhood.

Same shape of guarantee as *k*-means

E never increases, the number of medoid sets is finite ($\binom{n}{k}$), so PAM terminates — at a **local** optimum. Both methods are hill-climbers.

Robustness, measured rather than asserted

Two clean blobs of 150 points at (0,0) and (6,0), contaminated with a few points scattered in $[20, 45]^2$. “Centre error” is the largest distance from a true centre to the nearest thing the method returned.

outliers	k-means: centre error	ARI		k-medoids: medoid error	ARI
0	0.0951	1.000		0.0905	1.000
2	0.5210	1.000		0.0905	1.000
5	3.0826	0.000		0.0905	1.000
10	3.0826	0.000		0.0905	1.000
20	3.0826	0.000		4.0563	0.000

Five outliers out of 305 points destroy *k*-means: it spends a centre on them and merges the two real clusters, ARI 1.000 $\rightarrow$ 0.000. Two are already enough to drag a centre five times further off. *k*-medoids does not move until there are 20. **Robust, not immune.**

And it runs where *k*-means cannot run at all

Eight words, Levenshtein edit distance, $k = 2$. There is no arithmetic mean of two words, so *k*-means is simply not defined here.

```
medoids: ['cluster', 'density']  E = 19.0
cluster 1: cluster, clusters, clustered
cluster 2: medoid, medoids, density, densities, dense
```

n	k	PAM time (s)	<i>k</i> -means time (s)	ratio
250	4	0.0016	0.0037	0.4x
500	4	0.0100	0.0049	2.1x
1000	4	0.0378	0.0054	7.0x
2000	4	0.1817	0.0066	27.6x
4000	4	1.1032	0.0073	150.3x

fitted exponent for PAM, $t = c n^p$: $p = 2.30$
 exact: PAM needs $n(n-1)/2 = 7,998,000$ distances before its first swap;
k-means needs $n*k = 16,000$ per iteration. A factor of 500.

Faster than *k*-means at $n = 250$; two orders of magnitude slower at $n = 4000$, growing like $n^{2.3}$.
 Quote the count, not the clock: $500\times$ **the distances** before the first swap.

Predict first #5 — PAM on a hundred thousand rows

Commit to two numbers

You have 100,000 records and you want $k = 5$ medoids.

How many candidate swaps does *one* SWAP iteration consider? And how big is the dissimilarity matrix PAM needs?

Predict first #5 — PAM on a hundred thousand rows

Commit to two numbers

You have 100,000 records and you want $k = 5$ medoids.

How many candidate swaps does *one* SWAP iteration consider? And how big is the dissimilarity matrix PAM needs?

$$k(n - k)^2 = 5 \times 99995^2 = \mathbf{5.0 \times 10^{10}}$$
 evaluations per iteration.

The $n \times n$ float64 matrix is **74.5** GiB.

And PAM does not run for one iteration. **Both numbers have to come down, and the only lever available is n .**

CLARA, in six lines

```
Input : n objects, k, a sample size s (Kaufman and Rousseeuw suggest 40 + 2k)
best = None
repeat 5 times:
    S = a random sample of s objects from the data
    M = PAM(S, k) # only s x s dissimilarities needed
    cost = sum over ALL n objects of d(object, nearest medoid in M)
    if cost < best.cost: best = (M, cost)
return best
```

The complexity, and why it works

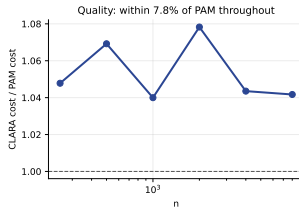
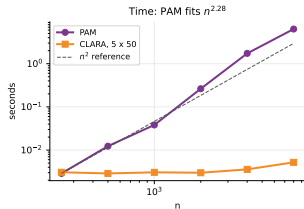
$O(ks^2 + k(n - k))$ per sample, so **linear in n** , and the matrix is $s \times s$, not $n \times n$. The evaluation step touches all n objects, which is what stops a lucky sample from looking better than it is.

Two honest limitations

1. CLARA can only return medoids that appeared in one of its samples. 2. It is randomised, so two runs give two answers.

Measured: PAM against CLARA on the same data

n	PAM time	PAM cost	CLARA time	CLARA cost	cost ratio	speed-up
500	0.012	661.537	0.002	707.320	1.0692	5.2x
1000	0.033	1290.963	0.003	1342.576	1.0400	12.4x
2000	0.268	2636.569	0.004	2842.923	1.0783	66.1x
4000	1.731	5261.167	0.004	5490.189	1.0435	421.0x



CLARA's time barely moves, because the PAM it runs is always on 50 objects. Only the final assignment sweep grows with n .

The trade, in two tables

$n = 4000$, $k = 5$. PAM reference: cost 5261.167 ARI 0.8404

sample size	samples	CLARA cost	cost/PAM	ARI	time (s)
20	5	5915.642	1.1244	0.8008	0.003
50	5	5490.189	1.0435	0.8266	0.004
250	5	5319.240	1.0110	0.8384	0.015
500	5	5302.009	1.0078	0.8302	0.048

CLARA over 10 seeds: cost 5421.410 to 5708.138 (spread 5.3%),
ARI 0.8052 to 0.8290. PAM is deterministic.

Buy quality with sample size

$s = 20$ costs 12% more than PAM; $s = 500$ costs 0.8% more and is still far faster than PAM at this n .

Say this out loud in the lab

CLARA gives a **different answer on every seed**. Report the seed, or run several and keep the best cost — which is what CLARA is already doing internally, five times.

One thing CLARA is not guilty of

6040 points: two clusters of 3000 and one of 40. A sample of 50 contains, on average, 0.33 points from the small cluster.

```
PAM medoids at [[9.44, 0.43], [10.62, -0.51], [0.04, 0.02]] cost 7239.719
cost if a medoid is forced onto each true centre: 7502.069
CLARA sample 50 cost 7269.834 ratio 1.0042 ARI 0.7426
CLARA sample 1000 cost 7254.729 ratio 1.0021 ARI 0.7457
```

Read the second line

PAM itself puts two medoids in one big cluster and none on the 40, and the “correct” solution is genuinely more expensive: 7502 against 7240. So this is a property of the **objective**, not of the sampling. *Minimising total dissimilarity does not reward small clusters.*

A completely different definition of “cluster”

Partitioning says

A cluster is **the set of points nearest to a representative**. Consequences: convex, you must supply k , every point is in a cluster.

Density says

A cluster is a **connected region where the points are packed closely together**, separated from other such regions by sparse space. Consequences: any shape, k discovered, and points in the sparse space belong to **nothing**.

DBSCAN — Density-Based Spatial Clustering of Applications with Noise, Ester, Kriegel, Sander and Xu, KDD 1996. Two parameters, no k , and an explicit category for “this point is not in any cluster”.

The definitions, precisely

Two parameters

ϵ (eps), a radius; and *minPts* (min_samples), a count. The ϵ -**neighbourhood** of p is $N_\epsilon(p) = \{q : d(p, q) \leq \epsilon\}$, including p itself in scikit-learn's convention.

Core

$|N_\epsilon(p)| \geq \text{minPts}$. The point sits in a dense region.

Border

Not core, but lies inside $N_\epsilon(q)$ for some *core* q . On the edge of a dense region.

Noise

Neither. Belongs to no cluster and is reported as -1.

Density-reachability, which is what glues a cluster together

q is **directly density-reachable** from a core point p if $q \in N_\epsilon(p)$. q is **density-reachable** from p if there is a chain $p = p_1, \dots, p_m = q$ with each p_{i+1} directly density-reachable from p_i (so $p_1 \dots p_{m-1}$ are all core). Two points are **density-connected** if both are density-reachable from a common core point. **A cluster is a maximal density-connected set.**

Eleven points, $\varepsilon = 1.5$, minPts = 3

$Q_1(1, 1)$ $Q_2(2, 1)$ $Q_3(1, 2)$ $Q_4(2, 2)$ $Q_5(3, 3)$ $Q_6(7, 3)$ $Q_7(8, 3)$ $Q_8(8, 4)$ $Q_9(9, 5)$ $Q_{10}(5, 6)$ $Q_{11}(11, 1)$

	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	Q11
Q1	0.000	1.000	1.000	1.414	2.828	6.325	7.280	7.616	8.944	6.403	10.000
Q2	1.000	0.000	1.414	1.000	2.236	5.385	6.325	6.708	8.062	5.831	9.000
Q3	1.000	1.414	0.000	1.000	2.236	6.083	7.071	7.280	8.544	5.657	10.050
Q4	1.414	1.000	1.000	0.000	1.414	5.099	6.083	6.325	7.616	5.000	9.055
Q5	2.828	2.236	2.236	1.414	0.000	4.000	5.000	5.099	6.325	3.606	8.246
Q6	6.325	5.385	6.083	5.099	4.000	0.000	1.000	1.414	2.828	3.606	4.472
Q7	7.280	6.325	7.071	6.083	5.000	1.000	0.000	1.000	2.236	4.243	3.606
Q8	7.616	6.708	7.280	6.325	5.099	1.414	1.000	0.000	1.414	3.606	4.243
Q9	8.944	8.062	8.544	7.616	6.325	2.828	2.236	1.414	0.000	4.123	4.472
Q10	6.403	5.831	5.657	5.000	3.606	3.606	4.243	3.606	4.123	0.000	7.810
Q11	10.000	9.000	10.050	9.055	8.246	4.472	3.606	4.243	4.472	7.810	0.000

Only two values matter against $\varepsilon = 1.5$: 1.000 and $\sqrt{2} = 1.414$ are in; 2.000 and $\sqrt{5} = 2.236$ are out.

Predict first #6 — classify three of them

Commit to three answers

$\epsilon = 1.5$, $\text{minPts} = 3$, counting the point itself.

Is $Q_5(3, 3)$ core, border or noise?

Is $Q_9(9, 5)$?

Is $Q_{10}(5, 6)$?

Predict first #6 — classify three of them

Commit to three answers

$\varepsilon = 1.5$, $\text{minPts} = 3$, counting the point itself.

Is $Q_5(3, 3)$ core, border or noise?

Is $Q_9(9, 5)$?

Is $Q_{10}(5, 6)$?

$N_\varepsilon(Q_5) = \{Q_5, Q_4\}$, size $2 < 3$: **not core**. But $Q_5 \in N_\varepsilon(Q_4)$ and Q_4 is core, so Q_5 is **BORDER**.

$N_\varepsilon(Q_9) = \{Q_9, Q_8\}$, size 2; Q_8 is core, so Q_9 is **BORDER**.

$N_\varepsilon(Q_{10}) = \{Q_{10}\}$, size 1, and it is in nobody else's ball: **NOISE**.

The commonest mistake is forgetting the point counts itself. With $\text{minPts} = 3$ a core point needs **two** other points within ε , not three.

By hand: every point, arithmetically

point	coord	N_eps(p)	N	verdict
Q1	(1, 1)	{Q1,Q2,Q3,Q4 }	4	CORE
Q2	(2, 1)	{Q1,Q2,Q3,Q4 }	4	CORE
Q3	(1, 2)	{Q1,Q2,Q3,Q4 }	4	CORE
Q4	(2, 2)	{Q1,Q2,Q3,Q4,Q5 }	5	CORE
Q5	(3, 3)	{Q4,Q5 }	2	BORDER
Q6	(7, 3)	{Q6,Q7,Q8 }	3	CORE
Q7	(8, 3)	{Q6,Q7,Q8 }	3	CORE
Q8	(8, 4)	{Q6,Q7,Q8,Q9 }	4	CORE
Q9	(9, 5)	{Q8,Q9 }	2	BORDER
Q10	(5, 6)	{Q10 }	1	NOISE
Q11	(11, 1)	{Q11 }	1	NOISE

core 7, border 2, noise 2

Q₆ and Q₇ each have exactly 3: **core by one point**. Change minPts to 4 and the right-hand cluster loses its core.

By hand: chain the core points into clusters

```
cluster 1 grown from Q1: core points {Q1,Q2,Q3,Q4}, then border Q5 joins
cluster 2 grown from Q6: core points {Q6,Q7,Q8}, then border Q9 joins
cluster 1 = {Q1,Q2,Q3,Q4,Q5}
cluster 2 = {Q6,Q7,Q8,Q9}
noise     = {Q10,Q11}
```

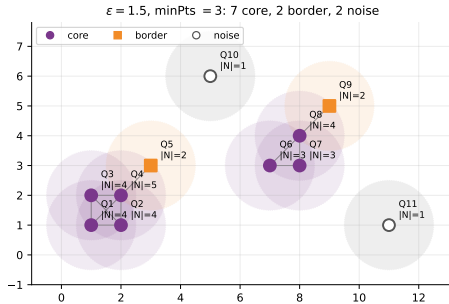
The procedure, exactly

Take any unvisited core point, take everything density-reachable from it — that is one cluster. Repeat. Whatever is left over is noise. Border points are attached to **whichever core cluster reached them first**.

The one non-determinism in DBSCAN

A border point within ϵ of two different clusters is assigned by visiting order. Core points and noise are never ambiguous. Here no border point is contested.

Drawn: the eleven points with their ϵ -balls



Circles are the ϵ -balls; grey edges join pairs within ϵ . Squares are border points, hollow circles are noise. Note that Q_{10} is not far from the data — it is 3.606 from both clusters — but $\epsilon = 1.5$ and that is all that matters.

In code: the library agrees exactly

```
db = DBSCAN(eps=1.5, min_samples=3).fit(P11)
print(db.labels_, sorted(db.core_sample_indices_))
```

```
sklearn labels_      [ 0  0  0  0  0  1  1  1  1 -1 -1]
hand labels          [ 0  0  0  0  0  1  1  1  1 -1 -1]
agree (up to renaming): True
sklearn core points   ['Q1', 'Q2', 'Q3', 'Q4', 'Q6', 'Q7', 'Q8']
hand core points      ['Q1', 'Q2', 'Q3', 'Q4', 'Q6', 'Q7', 'Q8']
identical: True
```

Two conventions worth writing down

labels_ uses **-1 for noise** — it is not a cluster index, and np.bincount on it will fail.
min_samples **counts the point itself**.

Both parameters change the answer

```
min_samples = 2 -> 2 clusters, 2 noise, labels [0,0,0,0,0,1,1,1,1,-1,-1]
min_samples = 3 -> 2 clusters, 2 noise, labels [0,0,0,0,0,1,1,1,1,-1,-1]
min_samples = 4 -> 2 clusters, 2 noise, labels [0,0,0,0,0,1,1,1,1,-1,-1]
min_samples = 5 -> 1 clusters, 6 noise, labels [0,0,0,0,0,-1,-1,-1,-1,-1,-1]

eps = 1.0 -> 2 clusters, 4 noise, labels [0,0,0,0,-1,1,1,1,-1,-1,-1]
eps = 1.4 -> 2 clusters, 4 noise, labels [0,0,0,0,-1,1,1,1,-1,-1,-1]
eps = 1.5 -> 2 clusters, 2 noise, labels [0,0,0,0,0,1,1,1,1,-1,-1]
eps = 2.5 -> 2 clusters, 2 noise, labels [0,0,0,0,0,1,1,1,1,-1,-1]
eps = 4.5 -> 1 clusters, 0 noise, labels [0,0,0,0,0,0,0,0,0,0,0]
```

Going from $\varepsilon = 1.4$ to 1.5 crosses $\sqrt{2}$ and turns two noise points into border points. Going from minPts 3 to 4 destroys the right-hand cluster, whose core points had exactly three neighbours.

Both parameters are cliffs, not dials.

What DBSCAN buys you: shape, *k*, and noise

data	eps	minPts	ARI	clusters	noise	(<i>k</i> -means ARI)
two moons	0.20	5	1.000	2	0	0.274
concentric circles	0.18	5	1.000	2	0	-0.002
two long thin bands	0.60	5	1.000	2	0	-0.001
very unequal sizes	0.85	5	0.857	3	19	0.288

blobs on a circle, eps = 0.45 and minPts = 5 held FIXED throughout:

```

true k = 2 -> DBSCAN found 2   ARI 0.9265   noise 9
true k = 3 -> DBSCAN found 3   ARI 0.9537   noise 11
true k = 5 -> DBSCAN found 5   ARI 0.9611   noise 18
true k = 8 -> DBSCAN found 8   ARI 0.9667   noise 26

```

ARI 1.000 on three shapes where *k*-means scores 0.27, -0.00 and -0.00 — and *k* **was never supplied**. *k*-means would have needed it told four times in the second block.

And it says “I do not know” — which *k*-means cannot

Three clean blobs of 300 points, plus 60 points scattered uniformly over the whole plot. $\epsilon = 0.55$, $\text{minPts} = 5$.

```
planted noise points: 60   DBSCAN called 57 points noise
of the planted noise, correctly flagged: 55 (91.7%)
of the genuine points, wrongly flagged:   2 ( 0.7%)
ARI on the 300 real points: DBSCAN 0.9901   k-means 1.0000
```

92% of the contamination identified, at the cost of two real points out of 300. That column of -1 s is a deliverable — outlier detection for free.

Read the ARI honestly

k-means scored *better* on the real points here, and it put every one of the 60 noise points inside a cluster. It has no vocabulary for “this belongs to nothing”.

Predict first #7 — two densities, one radius

Commit to an answer

Three clusters of 150 points. Two are **tight** (sd 0.12) and their centres are only 1.0 apart. The third is **loose** (sd 1.60) and sits well away from both.

Is there a value of ϵ that recovers all three?

Predict first #7 — two densities, one radius

Commit to an answer

Three clusters of 150 points. Two are **tight** (sd 0.12) and their centres are only 1.0 apart. The third is **loose** (sd 1.60) and sits well away from both.

Is there a value of ϵ that recovers all three?

No. At $\epsilon \leq 0.15$ the two tight clusters are perfect and **all** 150 points of the loose one are noise. By the time ϵ is large enough for the loose one to appear, the two tight ones have been **welded into a single cluster**.

ϵ is a *global* density threshold. One number cannot describe two densities. This is the honest, examinable weakness of DBSCAN.

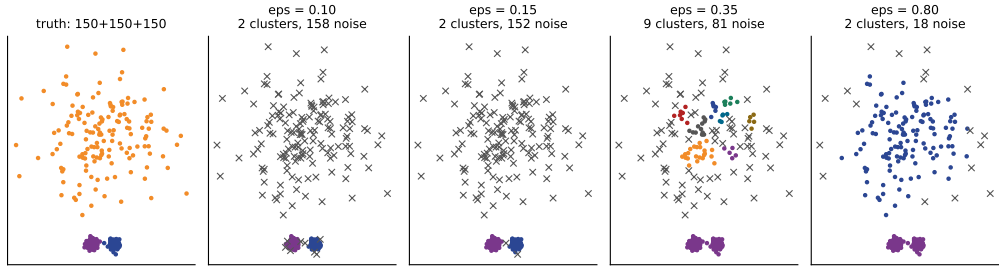
Measured: the whole sweep

eps	clusters	noise	tight A	tight B	loose C	
0.06	2	183	20 noise, 1 part	13 noise, 1 part	150 noise, 0 parts	
0.10	2	158	4 noise, 1 part	4 noise, 1 part	150 noise, 0 parts	
0.15	2	152	0 noise, 1 part	2 noise, 1 part	150 noise, 0 parts	
0.25	6	127	0 noise, 1 part	0 noise, 2 parts	127 noise, 4 parts	WELDED
0.35	9	81	0 noise, 1 part	0 noise, 1 part	81 noise, 8 parts	WELDED
0.50	4	54	0 noise, 1 part	0 noise, 1 part	54 noise, 3 parts	WELDED
0.80	2	18	0 noise, 1 part	0 noise, 1 part	18 noise, 1 part	WELDED
1.20	1	3	0 noise, 1 part	0 noise, 1 part	3 noise, 1 part	WELDED

“WELDED” means the two tight clusters have been merged into one. Between 0.15 and 0.25 the loose cluster is still 85% noise and the tight pair has already fused. **There is no window.**

And *k*-means with $k = 3$ scores ARI **0.3937** on the same data — it splits the loose cluster and merges the tight pair. Neither method is the answer here; **OPTICS** and **HDBSCAN** exist for exactly this.

Drawn: the same failure



Grey crosses are noise. Left to right: the truth, then $\epsilon = 0.10, 0.15, 0.35, 0.80$. Watch the top cluster appear only after the bottom two have merged.

Choosing ϵ : the *k*-distance graph

The standard recipe (Ester et al., §4.2)

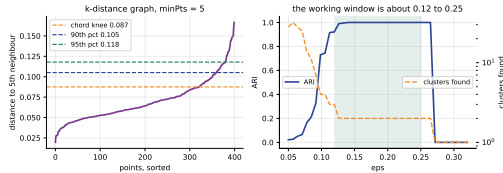
Fix minPts. For every point compute the distance to its (minPts – 1)-th nearest neighbour. Sort those *n* numbers and plot them. Points in a cluster have small values; noise points have large ones, so the curve has a **knee**. Read ϵ off the knee.

```
sorted 5th-nearest-neighbour distances, two moons, n = 400:  
  10% 0.0445   25% 0.0527   50% 0.0642   75% 0.0837  
  90% 0.1051   95% 0.1179   99% 0.1489  100% 0.1665  
automatic chord knee: 0.0875
```

The original paper says plainly that finding the knee automatically is “very difficult” and proposes an **interactive** procedure. Our automatic knee gives 0.0875 — and that is the wrong answer, as the next slide shows.

What each candidate ϵ actually gives

eps	clusters	noise	ARI	
0.060	31	121	0.031	
0.080	13	41	0.185	
0.087	7	24	0.320	<-- the automatic knee
0.100	4	10	0.731	
0.105	4	5	0.744	<-- 90th percentile
0.118	3	2	0.921	<-- 95th percentile
0.150	2	0	1.000	
0.200	2	0	1.000	
0.250	2	0	1.000	
0.270	1	0	0.000	
0.300	1	0	0.000	



So how sensitive is it, exactly?

The working window is a factor of two

0.12 to 0.25 gives $ARI \approx 1$. Just below it, 0.08 gives **13 clusters**. Just above it, 0.27 gives **1 cluster**. Between 0.25 and 0.27 the answer collapses completely.

What the graph is actually for

It converts “pick a positive real number” into “pick something between about 0.09 and 0.17” and then you look at the result. **It narrows the search. It does not end it.**

Rules of thumb worth remembering

$\text{minPts} \geq d + 1$, commonly $2d$; Ester et al. used 4 for two-dimensional data. Larger minPts means more noise and more robustness. **Scale your features first** — ϵ is a distance, so one column in rupees and one in metres makes it meaningless.

And in high dimensions, ϵ stops meaning anything

d	median 5-NN distance	ratio max/min NN distance
2	0.1742	300.4036
5	1.0580	9.6541
10	2.3147	3.2597
20	4.1659	2.4595
50	7.8523	1.6474
100	12.0239	1.3444

Read the right-hand column

In two dimensions the loneliest point is $300\times$ further from its neighbour than the most crowded one. At $d = 100$ that ratio is 1.34: **every point is about equally far from every other point**, so no threshold separates dense from sparse.

Measured on real data

With ϵ set automatically from the *k*-distance rule, DBSCAN scores ARI 0.5438 on iris ($d = 4$), -0.0074 on wine ($d = 13$), 0.0200 on breast cancer ($d = 30$) and 0.0002 on digits ($d = 64$), finding **one** cluster in the last three.

Which method, when

	<i>k</i> -means	<i>k</i> -medoids (PAM)	CLARA	DBSCAN
supply <i>k</i> ?	yes	yes	yes	no
representative	mean	a data object	a data object	—
dissimilarity	squared Euclid.	any	any	any
cluster shape	convex	convex	convex	arbitrary
outliers	drag the mean	robust-ish	robust-ish	labelled
deterministic	no	yes	no	yes*
time	$O(nkdi)$	$O(k(n-k)^2)/\text{iter}$	$O(ks^2 + k(n-k))$	$O(n \log n)$ indexed
memory	$O(nd)$	$O(n^2)$	$O(s^2 + n)$	$O(n)$
largest sane <i>n</i>	millions	thousands	millions	millions

measured at *n* = 200,000, *k* = 5:

k-means (*n*_init=10) 0.270 s

DBSCAN (eps = 0.30) 5.745 s

CLARA (5 x 50) 0.065 s

PAM: the distance matrix alone is 298.0 GiB

*except for border points equidistant from two clusters, which depend on visiting order.

Summary — fifteen lines

1. Partitioning asks for *k* and returns one partition. *k*-means minimises $WCSS = \sum_j \sum_{x \in C_j} \|x - c_j\|^2$.
2. Lloyd's algorithm: **assign to the nearest centroid, move each centroid to the mean of its cluster, repeat.**
3. Worked run: $123 \rightarrow 32.2 \rightarrow 16.74 \rightarrow 11.4167$, converged in three iterations, one point (P_3) changing cluster.
4. **Both steps can only lower WCSS** and there are finitely many partitions, so it stops — at a **local** optimum.
5. Twenty random starts on one data set: twelve different answers, worst $11.6\times$ the best. *k*-means++ reaches the best on 99.5% of single starts against 32% for random.
6. **Never report a single random start.** Use `n_init` and record the seed.
7. Elbow and silhouette are heuristics. On iris the chord elbow says 3, the drop-ratio elbow says 2, silhouette says 2, the species say 3. Uniform noise has a silhouette peak too.
8. *k*-means assumes **spherical, similar-sized, similar-density** clusters: ARI 0.274 on moons, -0.001 on two bands, and it cuts a 400/40/40 truth into 208/176/96.
9. **The mean is not robust**: one point at (12,9) moves a centroid by 2.15 and changes the partition; two outliers in 302 points take ARI from 1.000 to 0.000.
10. *k*-medoids uses an actual object and any dissimilarity, minimising $E = \sum d(x, o_j)$. Robust to those two outliers, and it clusters words by edit distance, where *k*-means is undefined.
11. **PAM** is BUILD then SWAP, $O(k(n-k)^2)$ per iteration and $O(n^2)$ memory: $500\times$ the distance computations of *k*-means before its first swap at $n = 4000$.
12. **CLARA** samples $40 + 2k$ objects five times, runs PAM on each, assigns all *n*, keeps the best: $1576\times$ less swap work at $n = 4000$ for 4.4% more cost. Randomised — 5.3% over ten seeds.
13. **DBSCAN**: core ($|N_c| \geq \text{minPts}$), border, noise; a cluster is a maximal density-connected set. On eleven points: 7 core, 2 border, 2 noise, 2 clusters, no *k* supplied.
14. DBSCAN gets ARI 1.000 on moons, circles and bands, discovers *k*, and flags 95% of planted noise.
15. **One global ϵ cannot serve two densities**, the working window on moons is a factor of two wide, and in 30 or 64 dimensions the method finds one cluster.

Before the next lecture

Do this with a pen

Take six to eight points and $k = 2$. Choose two of them as initial centroids *badly*. Run *k*-means to completion: the distance table at every assignment step, the means at every update step, and the WCSS after every half step. Check it against `KMeans(init=..., n_init=1)`.

Then take ten to twelve points, pick ε and `minPts`, and label every point core, border or noise from the distance matrix. Check it against DBSCAN.

Read

The notes: both hand runs are written out in full, and the practice questions include four more with complete solutions.

Assessment

A test on Unit VII follows this lecture, covering hierarchical, partitioning and density-based clustering together. The two hand calculations above are the numerically examinable core of it.

Next up

Consolidation across the whole course, and the remaining internal assessment.

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 10.; James et al., *An Introduction to Statistical Learning with Python*, Ch. 12.; Müller and Guido, *Introduction to Machine Learning with Python*, §3.5.; Ester, Kriegel, Sander and Xu, "A density-based algorithm for discovering clusters (DBSCAN)," KDD 1996.